

Lab 2 Report

Machine Learning for Hardware Design

Authors: Zach Dixon, Jongwoo Kim

List the different types of layers present

1. Convolutional layers with ReLU activation
2. Max Pooling layers
3. a Flatten layer
4. a Fully Connected layer with ReLU activation
5. a Fully Connected output layer with Softmax activation.

Write the mathematical formulation of each layer type**Include a brief description of each variable, its use/purpose, and how that is represented in our model**

Variable	meaning	Purpose	In Lab 1 model
X	Input feature map	Input data in current layer	ex) first Conv has 64*64*3
W	Kernel weight	Export input characteristics	ex) first Conv: 5*5*3*32
b_m	Bias	Added in each output channel	ex) 32 output filters -> add 32 bias
i, j	Spatial index	Location of feature map	ex) row, column of output image
c	Input channel	Choose input channel	ex) First Conv choose RGB 3 channel
m	Output channel	Choose filter for using	ex) First Conv choose 32 filters
Y	Output	Final output for current layer	ex) feature map after ReLU

1. Convolutional layers with ReLU activation

$$g_{i,j,m} = b_m + \sum_c \sum_p \sum_q X_{i+p,j+q,c} W_{p,q,c,m}$$

Convolution

$$Y_{i,j,m} = \max(0, g_{i,j,m})$$

ReLU

2. Max Pooling layers

$$Y_{i,j,c} = \max_{(p,q) \in R} X_{si+p,sj+q,c}$$

3. A Flatten layer

$$Y_k = X_{i,j,c}$$

4. A Fully Connected layer with ReLU activation

$$g_j = b_j + \sum_i X_i W_{i,j}$$

Fully connected layer

$$Y_j = \max(0, g_j)$$

ReLU

5. A Fully Connected output layer with Softmax activation

$$g_j = b_j + \sum_i X_i W_{i,j}$$

Fully connected

$$Y_i = \frac{e^{g_i}}{\sum_{l=0}^{N-1} e^{g_l}}$$

Softmax

Estimate the size of the memory needed for each layer and how many MAC computations are required for each layer

The memory requirement for each layer was estimated by summing the memory required for the input feature map, output feature map, weights, and biases. Since all data were stored as 32-bit floating-point values, each value required 4 bytes.

For convolutional layers, the number of multiply-accumulate (MAC) operations was estimated using:

$$\text{MACs} = \text{Hout} \times \text{Wout} \times \text{Cout} \times \text{Kh} \times \text{Kw} \times \text{Cin}$$

For fully connected layers, the MAC count was calculated using:

$$\text{MACs} = \text{Nin} \times \text{Nout}$$

Max pooling, flatten, and softmax layers were assigned 0 MAC operations because their primary operations are comparisons, data rearrangement, exponential calculations, additions, and divisions rather than multiply-accumulate operations. This does not mean that these layers require no computation.

Example Calculation: Conv1

Conv1 uses an input of $64 \times 64 \times 3$, an output of $60 \times 60 \times 32$, 32 kernels of size $5 \times 5 \times 3$, and 32 bias values.

Input memory:

$$64 \times 64 \times 3 \times 4 = 49,152 \text{ bytes}$$

Output memory:

$$60 \times 60 \times 32 \times 4 = 460,800 \text{ bytes}$$

Weight and bias memory:

$$(5 \times 5 \times 3 \times 32 + 32) \times 4 = 9,728 \text{ bytes}$$

Total memory:

$$49,152 + 460,800 + 9,728 = 519,680 \text{ bytes}$$

MAC operations:

$$60 \times 60 \times 32 \times 5 \times 5 \times 3 = 8,640,000 \text{ MACs}$$

Example Calculation: Dense1

Dense1 receives 2,048 input values and produces 256 output values.

Input memory:

$$2,048 \times 4 = 8,192 \text{ bytes}$$

Output memory:

$$256 \times 4 = 1,024 \text{ bytes}$$

Weight and bias memory:

$$(2,048 \times 256 + 256) \times 4 = 2,098,176 \text{ bytes}$$

Total memory:

$$8,192 + 1,024 + 2,098,176 = 2,107,392 \text{ bytes}$$

MAC operations:

$$2,048 \times 256 = 524,288 \text{ MACs}$$

Estimated Memory and MAC Operations for Each Layer

Layer	Input Memory (Bytes)	Output Memory (Bytes)	Weights + Biases (Bytes)	Total Memory (Bytes)	MACs
Conv1	49,152	460,800	9,728	519,680	8,640,000
Conv2	460,800	401,408	102,528	964,736	80,281,600
MaxPool 1	401,408	100,352	0	501,760	0
Conv3	100,352	173,056	73,984	347,392	12,460,032
Conv4	173,056	147,456	147,712	468,224	21,233,664
MaxPool 2	147,456	36,864	0	184,320	0
Conv5	36,864	25,600	147,712	210,176	3,686,400
Conv6	25,600	32,768	295,424	353,792	4,718,592
MaxPool 3	32,768	8,192	0	40,960	0
Flatten	8,192	8,192	0	16,384	0
Dense1	8,192	1,024	2,098,176	2,107,392	524,288
Dense2	1,024	800	205,600	207,424	51,200
Softmax	800	800	0	1,600	0

The complete model requires approximately 131.6 million MAC operations. Conv2 has the largest computational requirement with 80,281,600 MACs, accounting for the majority of the model's multiply-accumulate operations. In contrast, Dense1 requires relatively few MAC

operations compared with the convolutional layers but has the largest memory requirement because of its $2,048 \times 256$ weight matrix.

Verify that your outputs match the expected outputs sourced from the Tensorflow implementation

The outputs from the C++ implementation were compared with the expected outputs generated by the TensorFlow implementation. The provided comparison functions in the framework were used to evaluate the element-wise maximum error and cosine similarity between the two outputs.

Each layer was tested individually using the corresponding intermediate TensorFlow output as the expected result. The convolutional, max pooling, flatten, and first fully connected layers all produced outputs that closely matched the TensorFlow reference values.

The final end-to-end inference result, including the second fully connected layer and the Softmax activation, was also compared with the final TensorFlow output. The comparison produced the following result:

Element-wise Maximum Error: 2.38419×10^{-7}

Cosine Similarity: True, 100% (1.0)

The cosine similarity of 1.0 indicates that the C++ output and TensorFlow output were effectively identical in direction and relative values. The very small element-wise maximum error also shows that the numerical difference between the two implementations was negligible.

Therefore, the final C++ implementation successfully reproduced the expected TensorFlow inference output.

Layer Verification Results

Layer	Verification Result
Conv1	Passed
Conv2	Passed
MaxPool1	Passed

Conv3	Passed
Conv4	Passed
MaxPool2	Passed
Conv5	Passed
Conv6	Passed
MaxPool3	Passed
Flatten	Passed
Dense1	Passed
Dense2 + Softmax	Passed
Final End-to-End Inference	Passed

What was the final maximum error you saw/the error tolerance you used in your verification process for comparing your output? How did you come to that value and what is a reasonable error range? Where does this error come from and should there be any at all?

The final end-to-end maximum error observed between the C++ implementation and the TensorFlow reference output was 2.38419×10^{-7} . During individual layer verification, small numerical differences were also observed. For example, the Dense1 layer produced a maximum

error of 3.33786×10^{-6} . In both cases, the cosine similarity was 100% (1.0), indicating that the C++ output and TensorFlow output were effectively identical.

The framework uses an error tolerance of 0.001 for element-wise comparisons. The measured errors were much smaller than this tolerance, so the differences were considered acceptable. A reasonable error range for this implementation is therefore any small numerical difference that remains well below the selected tolerance and does not significantly affect the model output.

These small errors are expected because both TensorFlow and the C++ implementation use floating-point arithmetic. Floating-point operations can produce slightly different rounding results depending on the order of computations and the implementation of each operation. Because of this, exact equality between the two implementations is not always expected, even when they are mathematically equivalent.

Ideally, there would be no difference between the TensorFlow and C++ outputs. However, a very small non-zero error is normal for floating-point computation. Since the observed errors were on the order of 10^{-6} to 10^{-7} , remained far below the 0.001 tolerance, and produced 100% cosine similarity, the C++ implementation was considered to match the TensorFlow reference successfully.

Report the time taken for a single inference on both your x86 lab machine and the ZedBoard. Compare these to the time taken when a single inference is executed using the Tensorflow backend. You could use a Tensorboard to measure the performance. If you see a difference in execution time, why do you think there is a difference?

Platform / Backend	Input Image	Single Inference Time
TensorFlow	Image 0	66.444 ms
C++onx86 Lab Machine	Image 0	147.196 ms
C++ on ZedBoard	Image 0	2266.000 ms

The single-inference execution time was approximately 66.444 ms using the TensorFlow backend (measured with TensorBoard in Lab 1), 147.196 ms using the baseline C++ implementation on the x86 lab machine, and 2266.000 ms on the ZedBoard. The TensorFlow implementation was the fastest because it relies on highly optimized computational kernels and backend libraries that have been tuned for the underlying hardware. The baseline C++ implementation on x86 uses straightforward, unoptimized nested-loop implementations for each layer operation, which are designed primarily for correctness and readability rather than performance, resulting in a noticeably longer execution time than TensorFlow. The ZedBoard showed by far the highest latency, roughly 15 times slower than the x86 implementation, because

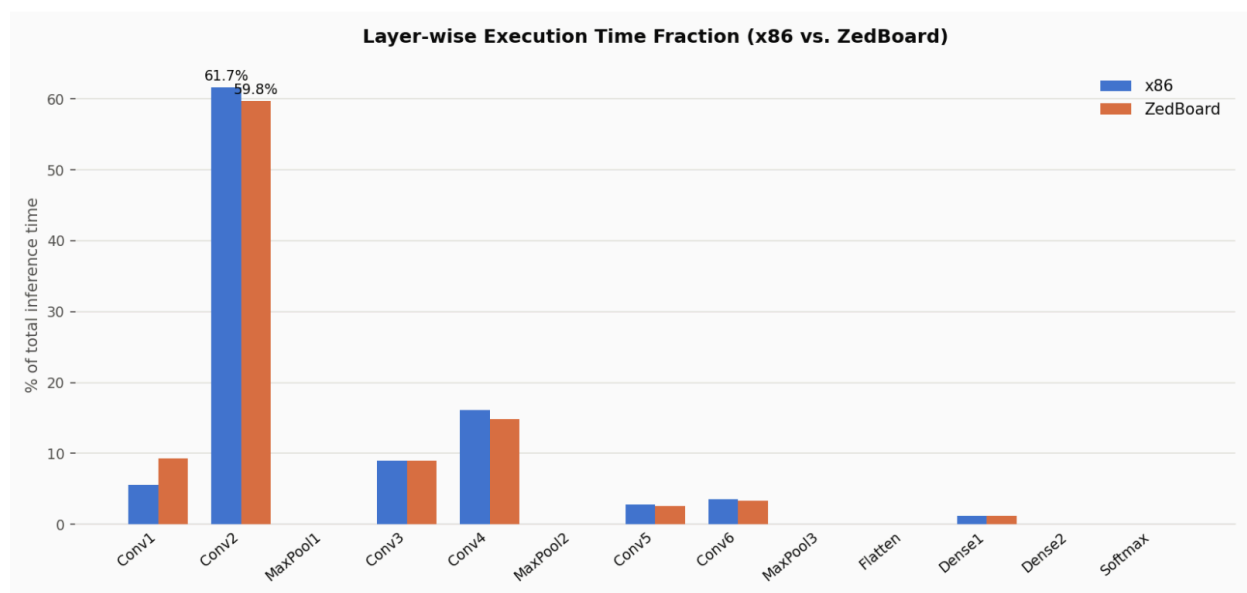
its embedded ARM Cortex-A9 processor has substantially fewer computational resources than the x86 lab machine: a lower clock frequency, a smaller cache hierarchy, and significantly lower memory bandwidth. Since the same unoptimized C++ implementation is used on both platforms, this large gap in execution time is attributable almost entirely to the difference in underlying hardware capability rather than to any difference in the algorithm itself.

Now using perf (for x86 as described in Section 3.5) and the Timer class (for the ZedBoard in src/Utils.h), figure out which layers takes most of the computation time. Plot the layerwise execution time fraction and report it. State possible reasons why a particular layer takes significant computation time? Does this match your Lab 1 results from Tensorboard? Which layers may be ripe for optimization? Why?

Layer	x86 Time (ms)	x86 (% of total)	ZedBoard Time (ms)	ZedBoard (% of total)
Conv1	7.977	5.58%	210.000	9.29%
Conv2	88.182	61.66%	1351.000	59.75%
MaxPool 1	0.044	0.03%	1.000	0.04%
Conv3	12.802	8.95%	203.000	8.98%
Conv4	23.006	16.09%	336.000	14.86%
MaxPool 2	0.014	0.01%	0.000	0.00%
Conv5	3.999	2.80%	58.000	2.57%
Conv6	5.127	3.59%	75.000	3.32%
MaxPool 3	0.002	0.00%	0.000	0.00%
Flatten	0.001	0.00%	0.000	0.00%
Dense1	1.783	1.25%	27.000	1.19%
Dense2	0.069	0.05%	0.000	0.00%
Softmax	0.002	0.00%	0.000	0.00%
Total	143.008	100%	2261.000	100%

The single-inference execution time was approximately 66.444 ms using the TensorFlow backend (measured with TensorBoard in Lab 1), 147.196 ms using the baseline C++ implementation on the x86 lab machine, and 2266.000 ms on the ZedBoard. TensorFlow was fastest because it uses optimized backend libraries, while our baseline C++ implementation uses unoptimized nested loops for each layer. The ZedBoard was roughly 15 times slower than the x86 implementation because its ARM Cortex-A9 processor has a lower clock frequency, a smaller cache, and lower memory bandwidth. Since the same unoptimized C++ code runs on both platforms, this gap is due to hardware, not the algorithm.

perf was not available in our lab environment (*perf: command not found*), so instead we used the framework's *Timer* class on both platforms — *std::chrono* on the x86 build and *XTime* on the ZedBoard build — to measure the wall-clock execution time of each individual layer. Using the same measurement method on both platforms allows a direct, consistent comparison between them.



On both platforms, the second convolutional layer (Conv2) dominates the total inference time, accounting for 61.66% on x86 and 59.75% on the ZedBoard. Conv3 and Conv4 add another ~20%, while pooling, flatten, and softmax are negligible.

Conv2 takes the most time because it has the largest number of multiply-accumulate (MAC) operations in the model — 80,281,600, as calculated in Section 3.3 — roughly nine times more than Conv1 and more than three times more than Conv4. This is because Conv2 operates on a 60×60 feature map with 32 input and 32 output channels, giving it a much larger computational footprint than the earlier layer (fewer channels) or the later layers (smaller spatial dimensions after pooling). Because our baseline C++ implementation uses unoptimized nested loops with no

vectorization or cache blocking, execution time scales almost linearly with MAC count, matching the layer ranking we observe on both x86 and the ZedBoard.

This result is also consistent with our Lab 1 TensorBoard profiling, where `conv2d_1` (corresponding to Conv2) was the single layer with the highest computation latency. The same layer dominating execution time across three different environments — TensorFlow, naive C++ on x86, and naive C++ on an embedded ARM core — suggests the bottleneck comes from the layer's algorithmic complexity (its MAC count), not from any platform-specific characteristic. Conv2 is therefore the clearest candidate for optimization, since even a modest speedup would reduce inference time the most; Conv3 and Conv4 would be the next-best targets. Suitable strategies include loop reordering and blocking/tiling for better cache locality, vectorizing the inner MAC loop with SIMD instructions, and multithreading the convolution across output channels.